

MalBERT: Malware Detection using Bidirectional Encoder Representations from Transformers*

Abir Rahali and Moulay A. Akhloufi, *Senior Member IEEE*

Abstract—In recent years we have witnessed an increase in cyber threats and malicious software attacks on different platforms with important consequences to persons and businesses. It has become critical to find automated machine learning techniques to proactively defend against malware. Transformers, a category of attention-based deep learning techniques, have recently shown impressive results in solving different tasks mainly related to the field of Natural Language Processing (NLP). In this paper, we propose the use of a Transformers architecture to automatically detect malicious software. We propose MalBERT, a model based on BERT (Bidirectional Encoder Representations from Transformers) which performs a static analysis on the source code of Android applications using preprocessed features to characterize existing malware and classify it into different representative malware categories. The obtained results are promising and show the high performance obtained by Transformer-based models for malicious software detection.

I. INTRODUCTION

Due to the exponential growth of digital equipment usage and easy access to internet technology, cyber threats to information systems such as computers and smartphones are increasing. Malicious software (Malware) is the primary tool used by attackers to perform cyber attacks. Different malware categories such as Trojans, Adwares, and Risktools are expeditiously developed and updated with recent encryption and deformation technologies to become a more severe threat to cyberspace. This rapid development incident often with harmful consequences to different data users at the level of both persons and businesses. For example, IBM reported that the average cost of a data breach is \$3.86 million as of 2020, however, the average time to identify a breach was 207 days [1]. As result, IT security professionals and researchers need to update the tools available to automatically detect new malware attacks.

A series of studies were conducted to solve this issue. Different analysis experiments were used on the static, dynamic and hybrid level [2] to extract various types of features such as binaries, permissions, and API calls. The features collected are then passed through detection models developed using machine learning and deep learning tools. The use of deep learning algorithms, for example, helped security specialists to analyze the most complex and targeted attacks.

*This work was enabled in part by support provided by the Natural Sciences and Engineering Research Council of Canada (NSERC), funding reference number RGPIN-2018-06233

A. Rahali and M.A. Akhloufi are with the Perception, Robotics, and Intelligent Machines Research Group (PRIME), Dept of Computer Science, Université de Moncton, Moncton, NB, Canada {ear4587, moulay.akhloufi}@umoncton.ca

The wildly used type of method is static analysis [3]. It is a known way of identifying malicious applications among benign applications, and this analysis focuses on the source code of software components that may be affected by malware. It is less expensive in terms of resources and time since no need to activate the malware by executing the code to capture the features, it can identify the maliciousness at the code level. For static analysis, there are mainly three practices to detect and classify malware: Permission-based (verify if the requested permissions are needed for the app to ensure the normal behavior on the app access and usage of the user data), Signature-based (identify if the app signature matches one of the malware signatures among the library collected in advance), and specification-based (verify if the app violates the rules set by experts to decide the maliciousness of a program under inspection).

Recent research in deep learning, led to the development of Transformer-based models such as BERT [4] and XLNet [5]. These approaches clearly showed impressive results in various state-of-the-art natural language processing (NLP) [6] and computer vision [7] tasks. Thanks to the attention mechanisms layers [8] added to the encoder-decoder architecture, Transformers can focus on the most important patterns in the data, leading to a remarkable boost in performance.

In this paper, we propose a malware detection approach using a Transformer-based algorithm. We experiment with different Transformer model architectures on our data. The dataset includes 11 different malware categories namely adware, spyware, ransomware, clicker, dropper, downloader, riskware, SMS-sender, horse-trojan, backdoor, and banker [9][10]. Our methodology focuses on the static analysis level on the source code of Android applications, to identify different categories of malware. Indeed, we did not limit the features to permission-based only but considered the whole software code as an important set of feature representations for the analysis. We started with training the model with the features after preprocessing, then a binary classification of the apps to malicious and benign, and finally a cross-category classification at the malware level.

II. BACKGROUND

Generally, sequence-to-sequence tasks are performed using an encoder-decoder model. RNN (Recurrent and Recursive Nets) architectures were the most widely-used architecture for both the encoder and decoder. But these architectures have some limitations.

A. RNN

RNN or also called sequence modeling like LSTM [11], is a family of neural networks for processing sequential data.

A recurrent network that maps an input sequence of x values to a corresponding sequence of output O values [11]. A loss L measures how far each O is from the corresponding training target y [11]. The loss L internally computes $y = \text{softmax}(O)$ and compares this to the target y [11].

In terms of limitations, RNN based architectures are hard to parallelize because the forward propagation graph is inherently sequential each time step may be computed only after the previous one, where both the runtime and the memory cost are $O(t)$ and cannot be reduced since states computed in the forward pass must be sorted until they are reused during the backward pass.

B. Encoder-Decoder

The seq2seq model normally has an encoder-decoder architecture [12], composed of an encoder that processes the input sequence and compresses the information into a context vector of a fixed length. This representation is expected to be a good summary of the meaning of the whole source sequence. And a decoder that is initialized with the context vector to emit the transformed output. The mean limitation of the encoder-decoder model is the disadvantage of this fixed-length context vector design and the incapability of remembering long sentences. While the decoder needs different information at different time steps.

C. Transformer

Thus, to solve the limitations of both RNN and encoder-decoder architectures, the authors of Transformers [8] have proposed a solution. They rely on the seq2seq encoder-decoder by replacing RNN with attention mechanisms. The attention mechanism allows the Transformers to have a very long-term memory. A Transformer model can "attend" or "focus" on all the previous tokens that have been generated. The attention mechanism allows the decoder to go back over the entire sentence and selectively extract the information it needs during decoding. Attention gives the decoder access to all the hidden states of the encoder. However, the decoder still has to make a single prediction for the next word, so we can not just pass a whole sequence to it (we have to pass it some kind of synthesis vector). So it asks the decoder to choose which hidden states to use and which to ignore by weighting the hidden states. The decoder then receives a weighted sum of hidden states to use to predict the next word.

In this section, we define the context of the Transformer-based approach by presenting most of the approaches related to their architectures. The Transformer in NLP is a new architecture that aims at solving sequence to sequence tasks while easily managing long-range dependencies. The Transformer has been proposed in [8]. A Transformer is an architecture that avoids recurrence and relies entirely on an attention mechanism to draw global dependencies between input and output. Prior to Transformers, dominant sequence

transduction models were based on complex recurrent or convolutional neural networks that include an encoder and a decoder.

Transformers also use an encoder and a decoder, but the elimination of recurrence in favor of attention mechanisms allows for much greater parallelization than methods such as RNNs and CNNs. The transformation is undoubtedly a huge improvement over the seq2seq models based on RNN. But it has its own set of limitations. Attention can only be paid to text strings of fixed length. The text must be divided into a number of segments or pieces before being introduced into the system as input and this causes context fragmentation. For example, BERT [4], a new linguistic representation model from Google AI, uses pre-training and fine-tuning to create state-of-the-art models for a wide range of tasks. These tasks include question answering systems, sentiment analysis, and linguistic inference. Transfer learning has been used in NLP using pretrained language models that have transformative architectures.

This implementation of BERT improves standard Transformers by removing the constraint of unidirectionality through the use of a pre-formed Masked Language Model (MLM) objective [13]. The Masked Language Model randomly masks certain elements of the input, and the objective is to predict the original vocabulary identifier of the masked word based solely on its context. Unlike pre-training the left to the right language model, the objective of the MLM allows the representation to merge the left and right context, which allows us to pre-train a deep bidirectional Transformer. In addition to the masked language model. BERT uses a Next Sentence Prediction task that jointly pre-trains the text pair representations.

Also, RoBERTa [14] an extension of BERT with some modifications in the pre-training procedure of the architecture, uses the same architecture. The modifications include training the model longer, with larger batches, and on a larger amount of data. The authors applied dynamic modification on the masking scheme [13] of the training data so the model can predict for longer sequences. They also collect a new dataset of comparable size to other privately used datasets to better control the effects of training set size.

In order to reduce the computation and training time of BERT, Sanh et al. proposed DistilBERT [15], a small, fast, cheap, and lightweight Transformer model based on the BERT architecture. This new extension uses knowledge distillation [16] during the pre-training phase to reduce the size of a BERT model by 40 %. To exploit the inductive biases learned by larger models during pre-training, the authors introduce a triple loss combining namely; language modeling loss, distillation loss, and cosine-distance loss.

Another approach based on the Transformer architecture is the autoregressive Transformer. XLNet [5], an example of this new implementation, exploits the best of modeling and automatic language encoding while trying to avoid its limitations. Instead of using a fixed forward or backward factorization order as in classical autoregressive models, XLNet maximizes the expected log probability of a sequence

with all possible permutations of the factorization order [6]. Also, thanks to the permutation operation usage, the context of each position can be made up of both left and right tokens, which enable XLNet to capture the bidirectional context information of all positions.

In this paper, we used the presented models in order to evaluate our proposed approach, by conducting a list of experiments on both binary and cross-category malware detection tasks.

III. RELATED WORKS

Over the past years, various approaches were proposed in deep learning to detect malware. These existing methods are mainly divided between; on one hand the training of recurrent neural networks (RNN), and convolutional neural networks (CNN), on a different set of extracted features, and on another hand the usage of attention mechanisms characteristics for malware classification.

A. Neural network-based approaches

Among these approaches, the study in [17] builds AMal-Net, a framework to learn multiple integration representations and family assignment with Graph CNN (GCNs) to model high-level graph semantics and use an Independent RNN (IndRNN) to decode deep semantic information. Seq-Mobile [18], is a behavior-based sequence approach that uses different recurrent neural networks (RNN). It extracts the semantic feature sequence, which can provide information of certain malicious behaviors, from binary files under a certain time constraint. Niu et al. [19], presents a new approach based on OpCode-level FCG. The FCG is obtained through static analysis of Operation Code (OpCode) using a Long Short-Term Memory (LSTM). The authors conduct experiments on a dataset on 1,796 Android malware samples classified into two categories and 1,000 benign Android apps. Jha et al. [20] focused on step size as an important factor concerning input size using RNN. They tested the model with three different feature vectors (hot-coding feature vector, random feature vector, and Word2Vec feature vector) using hyperparameters. Farid et al. [21] transform the android package kit (APK) file into a lightweight RGB image using a predefined dictionary and intelligent mapping, then apply a CNN on the obtained images for malware family classification. Multiple other examples of DNN based approach [22], [23], have been developed, with varying the feature extraction, selection, and representation methods with the aim of boosting the detection results.

B. Approaches using Attention Mechanisms

While our approach is, to the best of our knowledge, the first study to implement Transformers directly on software applications and preprocessing features like text to detect malware, few approaches have attempted the first steps towards using new methods such as attention mechanisms.

For example, Chen et al. [24] propose SLAM, a malware detection framework build based on the characteristics of the attention mechanism and the sliding window method. It uses

a feature extraction method of the API execution sequence according to its semantics. [25] uses a residual attention-based mechanism, based on this study the proposed method outperformed traditional CNN models. Similarly, Zhang et al. [26] propose a static analysis framework based on a set of N-gram opcodes sequence patches with a self-attention-based CNN named SA-CNN. Another example proposed by Yakura et al. [27], applies a CNN with an attention mechanism to images converted from binary datasets, by calculating an attention map to extract characteristic byte sequence. The distinction of regions in the attention map shows regions having higher importance for classification in the image.

In our work, we propose a novel approach based on BERT to better detect malware. We refined the pre-trained model to efficiently learn representations of source code language syntax and semantics. Our Context-Aware network learns contextual characteristics from a natural language sentences perspective thanks to the attention mechanism layers in the Transformer-based architecture.

IV. METHODOLOGY

This section explains the overall process of malware detection. The core idea of this work is to create, a malware detection framework using a Transformer-based approach. To reach this goal, we conducted a static analysis on the collected corpora from a natural language sentences perspective. So, we need a dataset including source code files and different categories of malware types. Figure 1 explains the logical flow of our Android malware detection. This Process is mainly divided into 4 phases. First, the Android files collection, then the Decompilation phase of the APK files, Feature Mining, and finally Deep Learning (DL) models training experiments.

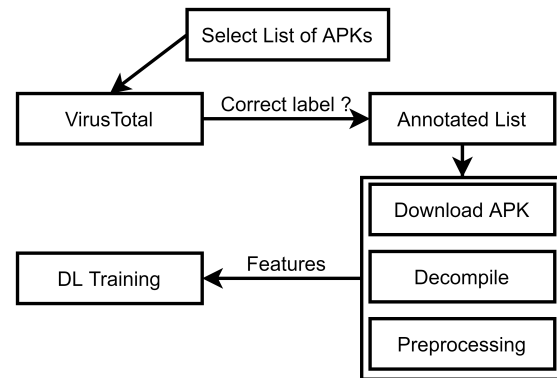


Fig. 1. Overview of the Methodology Steps

A. Data Collection

We collected the Android applications from the Androzoo public dataset. Androzoo, one of the state-of-the-art android malware datasets [28], is a growing collection of Android applications from several sources, including the official Google Play app market. It currently contains 13,320,014 different APKs, each of which has been analyzed by dozens of different antivirus products to find out which applications

are detected as malware. This public data is up to date with weekly analysis on the samples [29]. The data is labeled based on these analyses into malware and benign, and different malware categories and families.

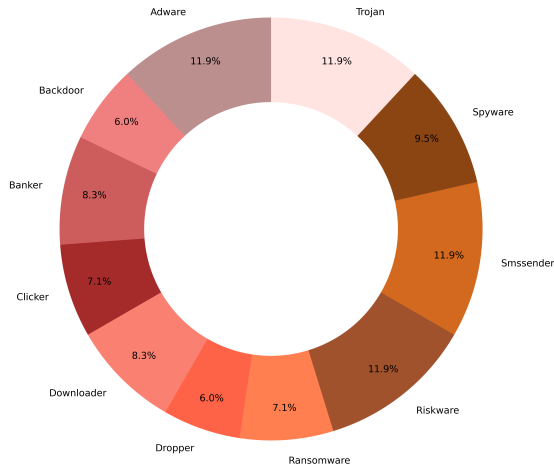


Fig. 2. Android Malware Categories

Based on state-of-the-art taxonomies for Android malware categories [30][31], we selected 11 categories namely; adware (displays advertising and entice a user to install it on their device), spyware (installs itself on the user device with the aim of collecting and transferring information without the user is aware of it), ransomware (takes personal data hostage), clicker (a type of trojan that performs a form of ad fraud. These “clickers” continuously make connections to websites, consequently awarding threat actors with revenue on pay-per-click bases), dropper (a syringe program or dropper virus, is a computer program created to install malicious software on a target system), downloader (a type of Trojan horse that downloads and installs malicious files), riskware (a software whose installation can represent a risk for the security of the computer, but however, not inevitably), SMS-sender (presents itself as a regular SMS messaging application and uses its basic permissions to send/receive short messages), horse-trojan (is designed to damage, disrupt, steal, or in general inflict some other harmful action on your data or network), backdoor (when introduced into the device, usually without the user’s knowledge, turns the software into a Trojan horse), and banker (is designed to steal data from users’ online bank accounts as well as data from online payment systems and plastic card systems). Figure 2 represents the distribution of these categories in our dataset. We select the list of APKs to download based on the recent creation and analysis date, then re-analyze this list with VirusTotal [32], to finally create our dataset list including 12,000 benign apps and 10,000 malware apps.

B. Preprocessing and Feature representations

Once the list of APKs is defined, we write a script to download the files. Then, we decompiled the downloaded APKs using Jdax [33], which creates folders of the apps’

files [34]. We extracted the Manifest.xml file from each sample. This file presents essential information about the application including the list of permissions, the activities, services, broadcast receivers, content providers, the version, and the meta-data. These files are then parsed to text format and passed through the preprocessing phase. In this step, we apply specific cleaning of the not important, mostly repeated words. We manually analyzed different examples and created a list of words and expressions that do not provide additional information. The purpose of the preprocessing is to reduce the input size to the limit of tokens specified by the Transformer. The final dataset format has 4 columns, the ID column, represented by the APK hash name, the Text column representing the Manifest files after preprocessing, the Label column, a binary format equal to 1 if the app is malware and 0 if not, and finally the Category column representing the malware type name (exp: adware).

C. Proposed Approach

Once the data is created and annotated in the right format, we split the data into train and test. We conduct all our experiments using BERT, we fine-tuned it on our train dataset. We fixed the hyperparameters based on each classification type. We train BERT to predict Malware/Benign (i.e., binary classification) for each sample, then, to predict the categories of malware (i.e, multi-classification). The Transformer architecture has specific input formatting steps including the creation of special tokens and ids. We use the Transformers implementation of the hugging face library [35] for the binary classification of Android applications. Only the Transformer architecture, layers, and weights are implemented, while all data formatting must be done beforehand to be compatible with the Transformer. While most pre-trained Transformers have essentially the same steps. Here we test this approach with BERT. Figure 3 gives a detailed overview of our approach.

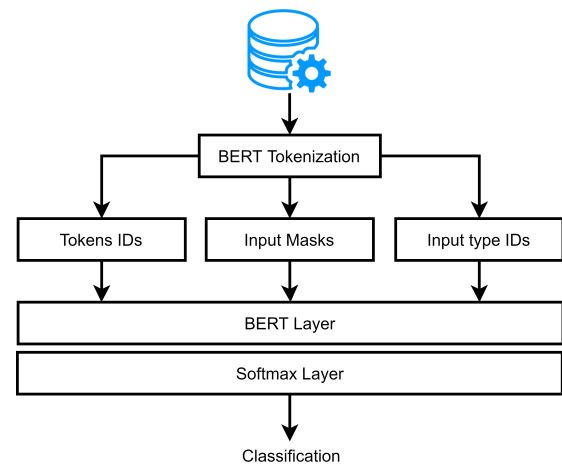


Fig. 3. BERT model fine-tuning process

V. EXPERIMENTAL RESULTS

To test the proposed approach, we evaluate it in terms of three main aspects: (1) the profitability on large and recent

categorical datasets, (2) the feature representation ability for information context extraction from android apps, and (3) the performance compared to the state-of-the-art approaches.

A. Parameters

The pretrained BERT base model (uncased) has 52000-word embeddings vocab size, 6 layers, and a position embeddings weight of 514. We kept all the parameters to these values and started the finetuning. In our finetuning implementation, we used Adam optimizer and set its hyperparameters as follow; the learning rate α to $2e-5$ for binary classification and $1e-3$ for multi-classification, $\alpha_1=0.9$ and $\alpha_2=0.999$. We set the max sequence size to 512 and the training batch size to 32. For feature creation, we used the BERT tokenizer, the Keras padding sequence then the BERT sequence for classification.

B. Baselines

We compare the proposed model with several state-of-the-art methods including LSTM [36]. We selected this sequencing model based on its performance in the reviewed studies. We also use XLNet [5], RoBERTa [14], and DistilBERT [15] as Transformer based baselines to compare their performance to BERT.

C. Evaluation Metrics

To evaluate the effectiveness of the model and avoid the contingency caused by the partitioning of the train and test sets, we select three metrics to evaluate the model (Accuracy, MCC, and Loss), which are commonly used in classification problems. We define the following numbers; TP (true positives), FN (false negatives), FP (false positives), and TN (true negatives).

The accuracy (ACC) is a metric for evaluating classification models. It is equal to the number of correct predictions divided by the total number of predictions, as equation 1 presents:

$$Accuracy = \frac{TP + TF}{TP + TF + FP + FN} \quad (1)$$

The Matthews Correlation Coefficient (MCC) is used for binary classification with an unbalanced dataset. It has a range of -1 to +1. We chose MCC over F1-score for binary classification as recommended in Chicco et al. [37]. MCC equation is defined as follows:

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (2)$$

The formula for the standard F1-score is the harmonic mean of the precision and recall. A perfect model has an F-score of 1. We used macro averaging in the results we presented in this paper. The formula of F1-score is defined as follows:

$$F1 = \frac{2 * TP}{2 * TP + FP + FN} \quad (3)$$

For the loss function, we used the cross-entropy loss, or log loss [38]. It measures the performance of a classification model whose output is a probability value between 0 and

1. Cross-entropy loss increases as the predicted probability diverge from the actual label. The cross-entropy $H(p, q)$ of the probability distribution p relative to a probability distribution q is given as:

$$H(p, q) = - \sum_{x \in \mathcal{X}} p(x) \log q(x) \quad (4)$$

D. Results and Analysis

We conducted the experiments on the preprocessed dataset. Fine-tuning the pre-trained models, clearly gave the highest accuracy results for this classification task compared to the LSTM baseline. The best classification model is BERT. The test metrics results of table I show that each Transformer learns differently depending on each architecture. The results in table I and table II prove that BERT outperformed the other baseline models in both binary and multi-classification malware detection. For BERT, the best learning rate shows that only two epochs are required before the loss starts to increase. Our fine-tuning with the training set included changing the hyperparameters to boost the results. To evaluate the final results, we used different evaluation metrics. The pretrained models achieved good results overall, but BERT obtained the best performance in both tasks.

TABLE I
DETECTION RESULTS USING THE FEATURE REPRESENTATION APPROACH
ACROSS DIFFERENT NETWORKS ON THE TEST DATASET FOR BINARY
CLASSIFICATION

Algos	ACC	F1	Loss	MCC
LSTM	0.9405	0.9382	0.1521	0.9077
XLNet	0.9579	0.9549	0.1461	0.9164
RoBERTA	0.9533	0.9499	0.1430	0.9070
DistilBERT	0.9542	0.9542	0.1305	0.9087
BERT	0.9761	0.9547	0.1274	0.9559

TABLE II
DETECTION RESULTS USING THE FEATURE REPRESENTATION APPROACH
ACROSS DIFFERENT NETWORKS ON THE TEST DATASET FOR
CROSS-CATEGORY MALWARE CLASSIFICATION

Algos	ACC	F1	Loss
LSTM	0.8507	0.7816	0.3521
XLNet	0.6232	0.7448	0.8106
RoBERTA	0.6491	0.7670	0.7611
DistilBERT	0.5981	0.7274	0.8505
BERT	0.9102	0.7804	0.2214

VI. CONCLUSION AND FUTURE WORKS

This paper studies malware classification using a novel approach based on Transformer. We detailed the malware text classification methodology and used it for feature representation. The BERT-based model achieved high accuracy results for both binary and cross-category classification, compared to the other baseline pre-trained language models. The results from the experiments show that the best binary accuracy is 0.9761 and for the multi-classification, it is 0.9102. We can conclude that the proposed approach results of feature representation as text input for a Transformer-based model,

are very interesting. So, the implementation of pre-trained linguistic models based on Transformer architectures in cybersecurity tasks can outperform standard RNN models like LSTM, when applied on a state-of-the-art dataset like Androzoo. Future work include testing MalBERT on other datasets, testing with other Transformers models such as GPT3, and creating an API to automatically detect malware in new applications.

REFERENCES

- [1] Jia Jen Low. Data breaches are getting more expensive – ibm security, 30 July 2020. Online: <https://techhq.com/2020/07/ibm-report-reveals-the-true-cost-of-a-data-breach/>, accessed 2021-03-31.
- [2] Anusha Damodaran, Fabio Di Troia, Corrado Aaron Visaggio, Thomas H Austin, and Mark Stamp. A comparison of static, dynamic, and hybrid analysis for malware detection. *Journal of Computer Virology and Hacking Techniques*, 13(1):1–12, 2017.
- [3] Ya Pan, Xiuting Ge, Chunrong Fang, and Yong Fan. A systematic literature review of android malware detection using static analysis. *IEEE Access*, 8:116363–116379, 2020.
- [4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.
- [5] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. In *Advances in neural information processing systems*, pages 5753–5763, 2019.
- [6] Yi Tay, Mostafa Dehghani, Dara Bahri, and Donald Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.
- [7] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. Transformers in vision: A survey. *arXiv preprint arXiv:2101.01169*, 2021.
- [8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
- [9] John Love. Malware types and classifications, MAR 28, 2018. <https://www.lastline.com/blog/malware-types-and-classifications/>, accessed 2021-03-31.
- [10] Beth Woodbury and Daniel Simpson. Malware names, 21 Jan 2021. Online: <https://docs.microsoft.com/en-us/windows/security/threat-protection/intelligence/malware-naming>, accessed 2021-03-31.
- [11] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [12] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112, 2014.
- [13] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. Mass: Masked sequence to sequence pre-training for language generation. *arXiv preprint arXiv:1905.02450*, 2019.
- [14] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [15] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
- [16] Mary Phuong and Christoph Lampert. Towards understanding knowledge distillation. In *International Conference on Machine Learning*, pages 5142–5151, 2019.
- [17] Xinjun Pei, Long Yu, and Shengwei Tian. Amalnet: A deep learning framework based on graph convolutional networks for malware detection. *Computers & Security*, page 101792, 2020.
- [18] Ruitao Feng, Jing Qiang Lim, Sen Chen, Shang-Wei Lin, and Yang Liu. Seqmobile: A sequence based efficient android malware detection system using rnn on mobile devices. *arXiv preprint arXiv:2011.05218*, 2020.
- [19] Weina Niu, Rong Cao, Xiaosong Zhang, Kangyi Ding, Kaimeng Zhang, and Ting Li. Opcode-level function call graph based android malware classification using deep learning. *Sensors*, 20(13):3645, 2020.
- [20] Sudan Jha, Deepak Prashar, Hoang Viet Long, and David Taniar. Recurrent neural network for detecting malware. *Computers & Security*, 99:102037, 2020.
- [21] Farid Naït-Abdesselam, Asim Darwaish, and Chafiq Titouna. An intelligent malware detection and classification system using apps-to-images transformations and convolutional neural networks. In *2020 16th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob)(50308)*, pages 1–6. IEEE, 2020.
- [22] Zhiqiang Wang, Qian Liu, and Yaping Chi. Review of android malware detection based on deep learning. *IEEE Access*, 8:181102–181126, 2020.
- [23] Francesco Mercaldo and Antonella Santone. Deep learning for image-based mobile malware detection. *Journal of Computer Virology and Hacking Techniques*, pages 1–15, 2020.
- [24] Jun Chen, Shize Guo, Xin Ma, Haiying Li, Jinhong Guo, Ming Chen, and Zhisong Pan. Slam: A malware detection method based on sliding local attention mechanism. *Security and Communication Networks*, 2020, 2020.
- [25] Shamika Ganesan, Moez Krichen, Roobaea Alroobaea, Soman KP, et al. Robust malware detection using residual attention network. *TechRxiv*. Preprint. <https://doi.org/10.36227/techrxiv.13385291.v1>, 2020.
- [26] Bin Zhang, Wentao Xiao, Xi Xiao, Arun Kumar Sangaiah, Weizhe Zhang, and Jiajia Zhang. Ransomware classification using patch-based cnn and self-attention network on embedded n-grams of opcodes. *Future Generation Computer Systems*, 110:708–720, 2020.
- [27] Hiromu Yakura, Shinnosuke Shinozaki, Reon Nishimura, Yoshihiro Oyama, and Jun Sakuma. Malware analysis of imaged binary samples by convolutional neural network with attention mechanism. In *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy*, pages 127–134, 2018.
- [28] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. Androzoo: Collecting millions of android apps for the research community. In *Proceedings of the 13th International Conference on Mining Software Repositories*, MSR '16, pages 468–471, New York, NY, USA, 2016. ACM.
- [29] Pei Liu, Li Li, Yanjie Zhao, Xiaoyu Sun, and John Grundy. Androzooopen: Collecting large-scale open source android apps for the research community. In *Proceedings of the 17th International Conference on Mining Software Repositories*, pages 548–552, 2020.
- [30] Najiahtul Syafiqah Ismail, Halizah Saad, Robiah Yusof, and Mohd Faizal Abdollah. General android malware behaviour taxonomy. *Science & Technology Research Institute for Defense (STRIDE)*, page 160, 2017.
- [31] Rashmi Rupendra Chouhan and Alpa Kavin Shah. A preface on android malware: Taxonomy, techniques and tools. *International Journal on Recent and Innovation Trends in Computing and Communication*, 5(6):1111–1117, 2017.
- [32] Peng Peng, Limin Yang, Linhai Song, and Gang Wang. Opening the blackbox of virustotal: Analyzing online phishing scan engines. In *Proceedings of the Internet Measurement Conference*, pages 478–485, 2019.
- [33] Emil Hozan. Android apk reverse engineering: Using jadx, October 4, 2019. Online: Android APK Reverse Engineering: Using JADX, accessed 2021-03-30.
- [34] Nicolas Harrand, César Soto-Valero, Martin Monperrus, and Benoit Baudry. Java decompiler diversity and its application to meta-decompilation. *Journal of Systems and Software*, page 110645, 2020.
- [35] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface’s transformers: State-of-the-art natural language processing. *ArXiv*, pages arXiv–1910, 2019.
- [36] Ralf C Staudemeyer and Eric Rothstein Morris. Understanding lstm—a tutorial into long short-term memory recurrent neural networks. *arXiv preprint arXiv:1909.09586*, 2019.
- [37] Davide Chicco and Giuseppe Jurman. The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation. *BMC genomics*, 21(1):6, 2020.
- [38] Zhilu Zhang and Mert Sabuncu. Generalized cross entropy loss for training deep neural networks with noisy labels. *Advances in neural information processing systems*, 31:8778–8788, 2018.